

Laravel

version: 12 latest

```
# create project
composer create-project laravel/laravel example-app // (menyesuaikan laravel yang
support dengan versi php yang sudah terinstall di pc/laptop)
composer create-project laravel/laravel:^12.0 example-app // (specifik ke versi
tertentu yang stabil terbaru)

# cek versi laravel
masuk ke directory project, jalankan php artisan --version

# Running web app
php artisan serve

# Membuat project under maintenance
php artisan down

# Mengembalikan project aktif kembali
php artisan up
```

Controller: **tempat khusus untuk menyimpan logika aplikasi.**

```
# Controller kosong
php artisan make:controller ProdukController

# Controller lengkap dengan 7 method CRUD sekaligus
php artisan make:controller ProdukController --resource

# Controller + otomatis buat Model sekaligus
php artisan make:controller ProdukController --resource --model=Produk

# Struktur Controller Dasar
// app/Http/Controllers/ProdukController.php

namespace App\Http\Controllers;
use Illuminate\Http\Request;

class ProdukController extends Controller
{
    // Method untuk menampilkan daftar produk
    public function index()
    {
        return view('produk.index');
    }

    // Method untuk menampilkan detail produk
    public function show($id)
    {
        return view('produk.show', compact('id'));
    }
}
```

Route

```
// routes/web.php
use App\Http\Controllers\ProdukController;

// Route paling sederhana
```

```
Route::get('/halo', function () {
    return 'Halo Dunia!';
    return view('beranda'); // jika dikirim ke suatu view

    return view('profil', [ // dikirim ke suatu view with data
        'nama' => 'Budi Santoso',
        'kota' => 'Surakarta',
    ]);

    // di view.blade.php
    <h1>{{ $nama }}</h1>    → Budi
    <p>{{ $kota }}</p>      → Surakarta
});
```

```
// with Controller
Route::get('/produk', [ProdukController::class, 'index']);
Route::get('/produk/{id}', [ProdukController::class, 'show']);
```

```
akses di view
{{-- Ke halaman list produk --}}
<a href="{{ url('produk') }}">Lihat Produk</a>
{{-- Ke halaman detail produk --}}
<a href="{{ url('produk', $produk->id) }}">Detail Produk</a>
```

```
Penambahan naming di Route
Route::get('/produk', [ProdukController::class, 'index'])->name('produk.index');
Route::get('/produk/{id}', [ProdukController::class, 'show'])->name('produk.show');
```

```
akses di view
{{-- Ke halaman list produk --}}
<a href="{{ route('produk.index') }}">Lihat Produk</a>

{{-- Ke halaman detail produk --}}
<a href="{{ route('produk.show', $produk->id) }}">Detail Produk</a>
```

Model

Model adalah **representasi PHP dari sebuah tabel database**. Setiap baris di tabel = satu objek Model di PHP. Lewat Model, kamu bisa baca, simpan, ubah, dan hapus data tanpa perlu nulis SQL manual.

```
// Perintah dasar membuat model
php artisan make:model Produk

// Kombinasi flag yang sering dipakai
# Model + Migration
php artisan make:model Produk -m

# Model + Migration + Controller Resource (favorit pemula)
php artisan make:model Produk -mcr

# Model + Migration + Controller + Factory + Seeder (lengkap)
php artisan make:model Produk -mcfers

# Semua sekaligus
php artisan make:model Produk --all
```

Struktur Model dasar

```
<?php
// app/Models/Produk.php

namespace App\Models;
```

```

use Illuminate\Database\Eloquent\Model;

class Produk extends Model
{
    // 1. Kolom yang boleh diisi massal
    protected $fillable = [
        'nama',
        'deskripsi',
        'harga',
        'stok',
    ];

    // 2. Cast tipe data otomatis (Secara default, semua data yang keluar dari database
    berbentuk **string** – tidak peduli tipe aslinya di tabel. `$casts` bertugas
    **mengkonversi otomatis** tipe data tersebut saat dibaca maupun disimpan.)
    protected $casts = [
        'harga' => 'decimal:2',
        'aktif' => 'boolean',
    ];
}

```

contoh penggunaan

```

// Semua produk
$produk = Produk::all();

// Satu produk by ID
$produk = Produk::find(1);

// Dengan kondisi
$murah = Produk::where('harga', '<', 50000)->get();

// Satu data atau langsung 404 jika tidak ada
$produk = Produk::findOrFail($id);

```

Contoh Model lengkap dengan relasi

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Produk extends Model
{
    protected $fillable = [
        'nama', 'deskripsi', 'harga', 'stok', 'aktif', 'kategori_id'
    ];

    protected $casts = [
        'harga' => 'decimal:2',
        'aktif' => 'boolean',
    ];

    // Relasi: produk milik satu kategori
    public function kategori(): BelongsTo
    {
        return $this->belongsTo(Kategori::class);
    }
}

```

```
// Relasi: produk punya banyak ulasan
public function ulasan(): HasMany
{
    return $this->hasMany(Ulasan::class);
}
}
```

Cara pakainya di Controller:

```
// Ambil produk beserta nama kategorinya (1 query, bukan N+1)
$produk = Produk::with('kategori')->get();

// Akses relasi
foreach ($produk as $p) {
    echo $p->nama;           // nama produk
    echo $p->kategori->nama; // nama kategori-nya
}
```

Kalau nama tabelmu tidak sesuai konvensi, bisa override:

```
class Produk extends Model
{
    protected $table = 'tbl_produk'; // nama tabel custom
}
```

Migration

Migration adalah fitur Laravel yang berfungsi sebagai **version control untuk database**. Artinya struktur tabel database ditulis dalam bentuk kode PHP, bukan membuat tabel langsung di phpMyAdmin

Daftar Perintah Migration

```
# Melihat status migration (sudah dijalankan atau belum)
php artisan migrate:status

# Membatalkan migration terakhir (rollback 1 batch)
php artisan migrate:rollback

# Rollback sejumlah batch tertentu
php artisan migrate:rollback --step=2

# Rollback SEMUA migration
php artisan migrate:reset

# Rollback semua lalu migrate ulang dari awal
php artisan migrate:refresh

# Hapus semua tabel lalu migrate ulang dari awal
php artisan migrate:fresh

# Migrate sekaligus isi data awal (seeder)
php artisan migrate --seed
php artisan migrate:fresh --seed
```

File akan terbuat di database/migrations/ :

```
public function up(): void
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
    });
}
```

```

        $table->string('name');
        $table->string('email')->unique();
        $table->string('image')->nullable();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('students');
}

```

****Fungsi up() dan down()**

```

// up() → dijalankan saat migrate (membuat tabel)
public function up(): void { ... }

// down() → dijalankan saat rollback (menghapus tabel)
public function down(): void { ... }

```

Menambahkan Seeder di Laravel

Seeder digunakan untuk mengisi database dengan data awal/dummy secara otomatis.

Langkah-langkahnya:

1. Buat file Seeder

```
php artisan make:seeder UserSeeder
```

2. Edit file seeder di database/seeder/UserSeeder.php

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\User;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name'      => 'Admin',
            'email'     => 'admin@example.com',
            'password' => Hash::make('password'),
        ]);
    }
}

```

3. Daftarkan di DatabaseSeeder.php

```

// database/seeder/DatabaseSeeder.php

public function run(): void
{
    $this->call([
        UserSeeder::class,
    ]);
}

```

```
    });  
}
```

4. Jalankan seeder

```
# Jalankan semua seeder  
php artisan db:seed  
  
# Jalankan seeder tertentu saja  
php artisan db:seed --class=UserSeeder  
  
# Reset + migrate + seed sekaligus  
php artisan migrate:fresh --seed
```

Alternatif lain menggunakan Factory di Seeder (data dummy massal)

```
php artisan make:factory UserFactory --model=User
```

```
// Di UserFactory.php  
public function definition(): array  
{  
    return [  
        'name' => fake()->name(),  
        'email' => fake()->unique()->safeEmail(),  
        'email_verified_at' => now(),  
        'password' => static::$password ??= Hash::make('password'),  
        'remember_token' => Str::random(10),  
    ];  
}
```

```
// Di UserSeeder.php  
public function run(): void  
{  
    // Buat 50 user dummy sekaligus  
    User::factory(50)->create();  
}
```

Perintah Artisan Storage di Laravel

Perintah Utama

```
# Membuat symlink storage ke public  
php artisan storage:link
```

Perintah ini membuat **symbolic link** dari:

```
public/storage → storage/app/public
```

Sehingga file yang diupload bisa diakses lewat browser:

```
http://localhost/storage/foto.jpg
```

Kenapa perlu `storage:link` ?

Karena folder `storage/` tidak bisa diakses langsung dari browser, maka perlu dibuat symlink ke folder `public/` yang bisa diakses publik.

```
// Menyimpan file
$path = $request->file('image')->store('images', 'public');

// Menampilkan file di blade

// alternatif
// tambahkan @use('Illuminate\Support\Facades\Storage') di view paling atas
image)

// ✅ cek file ada atau tidak
Storage::exists($student->image)

// ✅ ambil isi file
Storage::get($student->image)

// ✅ hapus file
Storage::delete($student->image)

// ✅ simpan file
Storage::put('images/foto.jpg', $file)

// ✅ download file
Storage::download($student->image)
```

Directive Blade di Laravel

Blade adalah template engine bawaan Laravel. Kamu bisa menulis HTML biasa, lalu tambahkan logika dengan direktif `@` yang mudah dibaca.

1. Kondisi (If/Else)

```
@if($user->isAdmin())
    <p>Admin</p>
@elseif($user->isMember())
    <p>Member</p>
@else
    <p>Guest</p>
@endif

{{-- Kebalikan if --}}
@unless($user->isAdmin())
    <p>Bukan Admin</p>
@endunless
```

2. Loop

```
{{-- Foreach --}}
@foreach($students as $student)
    <p>{{ $student->name }}</p>
@endforeach

{{-- Forelse (jika data kosong) --}}
@forelse($students as $student)
    <p>{{ $student->name }}</p>
@empty
    <p>Tidak ada data</p>
@endforelse

{{-- For --}}
@for($i = 0; $i < 10; $i++)
    <p>{{ $i }}</p>
@endfor

{{-- While --}}
@while($condition)
    <p>Looping...</p>
@endwhile
```

3. Loop Variable \$loop

```
@foreach($students as $student)
    {{ $loop->index }}      {{-- index mulai 0 --}}
    {{ $loop->iteration }}  {{-- iterasi mulai 1 --}}
    {{ $loop->count }}      {{-- total data --}}
    {{ $loop->first }}      {{-- true jika pertama --}}
    {{ $loop->last }}       {{-- true jika terakhir --}}
@endforeach
```

4. Layout & Section

```
{{-- Parent layout --}}
@yield('content')
@yield('title', 'Default Title')

{{-- Child blade --}}
@extends('layouts.app')

@section('title', 'Halaman Siswa')
```

```
@section('content')
    <p>Konten di sini</p>
@endsection
```

5. Component & Include

```
{{-- Include partial blade --}}
@include('partials.navbar')
@include('partials.alert', ['message' => 'Berhasil!'])

{{-- Component --}}
@component('components.alert')
    Isi konten component
@endcomponent
```

6. Auth

```
@auth
    <p>Sudah login: {{ auth()->user()->name }}</p>
@endauth

@guest
    <p>Belum login</p>
@endguest
```

7. CSRF & Method

```
{{-- Wajib di setiap form POST --}}
@csrf

{{-- Spoofing method (PUT, PATCH, DELETE) --}}
@method('PUT')
@method('DELETE')
```

8. Menampilkan Data

```
{{-- Escaped (aman dari XSS) --}}
{{ $student->name }}

{{-- Unescaped (HTML mentah) --}}
{!! $student->bio !!}

{{-- Komentar blade (tidak muncul di HTML) --}}
{{-- Ini komentar --}}
```

9. Stack & Push

```
{{-- Di layout --}}
@stack('scripts')

{{-- Di child blade --}}
@push('scripts')
    <script src="app.js"></script>
@endpush
```

10. Lainnya

```
{{-- Cek data ada atau tidak --}}
@isset($student)
    <p>{{ $student->name }}</p>
@endisset

{{-- Cek data kosong --}}
@empty($students)
    <p>Tidak ada data</p>
@endempty

{{-- Switch Case --}}
@switch($role)
    @case('admin')
        <p>Admin</p>
        @break
    @case('user')
        <p>User</p>
        @break
    @default
        <p>Guest</p>
@endswitch

{{-- Env --}}
@env('local')
    <p>Mode Development</p>
@endenv
```

Ringkasan

Kategori	Directive
Kondisi	@if @elseif @else @unless
Loop	@foreach @forelse @for @while
Layout	@extends @section @yield @stack
Include	@include @component
Auth	@auth @guest
Form	@csrf @method
Data	{{ }} {!! !!} @isset @empty

Penjelasan @stack dan @push

Analoginya

@stack adalah tempat/wadah, @push adalah yang mengisi wadah tersebut.

@stack — Tempat Penampung

Diletakkan di **layout utama**, sebagai tempat dimana konten akan dimunculkan.

```
{{-- resources/views/layouts/app.blade.php --}}
<html>
<head>
    <title>App</title>
</head>
<body>
    @yield('content')

    {{-- Wadah untuk script tambahan --}}
    @stack('scripts')    ← konten dari @push akan muncul di sini
```

```
</body>
</html>
```

@push — Yang Mengisi Wadah

Diletakkan di **child blade**, untuk mengirim konten ke `@stack` .

```
{{-- resources/views/students/index.blade.php --}}
@extends('layouts.app')

@section('content')
    <p>Halaman Siswa</p>
@endsection

{{-- Kirim script ke @stack('scripts') di layout --}}
@push('scripts')
    <script src="app.js"></script>
    <script>
        console.log('Halo dari halaman siswa!');
    </script>
@endpush
```

Hasil HTML yang dirender

```
<html>
<head>
    <title>App</title>
</head>
<body>
    <p>Halaman Siswa</p>

    <!-- konten dari @push muncul di sini -->
    <script src="app.js"></script>
    <script>
        console.log('Halo dari halaman siswa!');
    </script>
</body>
</html>
```

Kenapa tidak pakai @section saja?

```
{{-- @section hanya bisa diisi SEKALI --}}
@section('scripts')
    <script src="a.js"></script>
@endsection

{{-- @push bisa diisi dari BANYAK file blade --}}
@push('scripts')
    <script src="a.js"></script>
@endpush

@push('scripts')
    <script src="b.js"></script> ← ditambahkan, tidak menimpa
@endpush
```

Perbandingan @stack/@push vs @yield/@section

	@yield / @section	@stack / @push
Bisa diisi dari banyak tempat	✗ Tidak	✓ Ya
Menimpa konten sebelumnya	✓ Ya	✗ Tidak (ditambahkan)
Cocok untuk	Konten utama	Script / CSS tambahan

Kesimpulan

@stack dan @push cocok dipakai untuk **script atau CSS yang hanya dibutuhkan di halaman tertentu** saja, tidak semua halaman. Misalnya script datepicker hanya di halaman form, script chart hanya di halaman dashboard.